HoloRig

Gesture-Controlled Holographic Interaction System

Developed for: ORIGO 2025 Workshop

Author: Joshua Jacob Thomas

Institution: National Institute of Technology Calicut (NITC)

This report documents the design and implementation of HoloRig, a real-time gesture-controlled holographic interaction system built as a demo project for the ORIGO 2025 robotics and computer vision workshop.

Abstract

HoloRig is a real-time gesture-controlled holographic interaction system that combines computer vision, 3D rendering, and inverse kinematics in a single interactive hub. Using a standard webcam and a hand-tracking pipeline, the system allows users to rotate and zoom 3D holograms, navigate a gesture-only menu, and control a simulated 2-link robotic arm with a virtual gripper. All interaction happens through intuitive hand gestures, without the need for a mouse, keyboard, or additional hardware.

The main goal of this work was to build something that looks and feels like a futuristic control console, while remaining fully implementable with accessible tools such as Python, OpenCV, NumPy, and a MediaPipe-based hand tracking wrapper. HoloRig is intended as a live demo piece for workshops and events like ORIGO 2025, where participants can directly interact with the system.

1. Introduction

Gesture-based interfaces are becoming increasingly relevant in human computer interaction, especially in contexts where a hands free or touchless control scheme is useful. HoloRig was designed as an experimental prototype to explore this style of interaction. The project is implemented as a single Python application that opens a webcam feed, overlays a holographic themed UI, and lets the user move between multiple interactive demos purely through hand gestures.

Once the program starts, the user sees an intro screen, followed by a tile based menu. From there, they can launch two different 3D demos and a robotic arm demo. The same small set of gestures is reused across modes so that the learning curve stays low.

Key ideas explored in HoloRig include:

- Real time hand landmark tracking using cvzone and the MediaPipe Hands model.
- A compact custom 3D engine for rotating and projecting points in space.
- Gesture based control over rotation, zoom, and mode selection.
- A 2 link planar robotic arm simulated with inverse kinematics and a virtual gripper.
- A consistent holographic style UI to tie the demos together visually.

2. System Overview

At a high level, HoloRig is organised into five main subsystems. These run together in a loop for each frame captured from the webcam.

Subsystems:

- Camera and frame handling: capture, flip, resize, and darken the video feed to create space for graphics.
- Hand tracking: use cvzone.HandTrackingModule to obtain 21 landmarks per detected hand and basic finger states.
- Gesture processing: interpret distances and positions as pinch, open palm, two hand stretch, and palm tilt gestures.
- UI and mode management: draw the grid, HUD, titles, menu tiles, back buttons, and track which demo is active.
- Interaction modules: implement the gyro rings, orbit core, and robotic arm simulations.

For each frame, the pipeline can be summarised as: capture frame, preprocess, detect hands, derive gestures, update internal state variables (such as rotation angles and zoom), redraw the current mode, and display the resulting frame.

3. Gesture Recognition and Hand Tracking

HoloRig relies on deterministic geometry based rules instead of a separate machine learning model for gesture classification. Hand detection and landmark extraction are handled by cvzone and MediaPipe, after which the code computes simple distances and directions to recognise a small set of gestures.

Pinch detection is done by measuring the Euclidean distance between the index fingertip (landmark 8) and the thumb tip (landmark 4). If this distance is below a fixed pixel threshold, the system treats it as an active pinch. This gesture is used for selecting tiles, dragging 3D objects, and closing the robotic gripper.

When two hands are present, the distance between the two index fingertips is used to control zoom in the 3D modes. The code stores the initial distance as a baseline and adjusts a zoom factor based on how the distance changes. Moving the hands apart zooms in; bringing them together zooms out.

The palm tilt gesture is estimated using the horizontal offset between the wrist and the middle knuckle (landmarks 0 and 9). If that offset crosses a small threshold, the system applies a small continuous rotation around the Z axis to the 3D object, which feels like spinning the hologram by tilting the hand.

4. 3D Rendering and Projection

Instead of using a dedicated 3D engine, HoloRig uses a compact custom projection pipeline. The 3D objects are stored as arrays of points (x, y, z) and a list of edges that connect pairs of indices into line segments. Every frame, these points are rotated in 3D using rotation matrices and then projected onto the 2D window.

Rotation is performed by constructing three rotation matrices, one for each axis, based on the current rotation angles around X, Y, and Z. These matrices are multiplied together to form a single transform, and then applied to all points using matrix multiplication via NumPy.

For projection, the rotated X and Y coordinates are scaled by a factor that depends on the zoom and then translated so that the object appears centered in the window. The Z coordinate is used only for depth based brightness: segments that are farther away are drawn slightly dimmer, and closer segments appear brighter. This gives a reasonable sense of depth while keeping the math simple.

5. 3D Objects: Gyro Rings and Orbit Core

HoloRig includes two main 3D demos. The first is a set of gyroscope style rings, and the second is a structure called the orbit core. Both are generated procedurally by evaluating simple analytical formulas.

The gyro rings demo creates three circles in three perpendicular planes. Each ring is made by sampling points around a circle and setting the coordinates so that one axis stays fixed for each ring. Edges are then added between consecutive sample points to close the loops. When the user rotates the object, the rings tumble inside each other like a three axis gimbal.

The orbit core demo constructs two cubes. The inner cube and a larger outer cube are both represented by eight vertices each. Edges are created for each cube, and additional edges connect corresponding vertices between the inner and outer cubes. With depth shading and glow effects, this looks like a smaller core suspended inside a wireframe shell.

6. Robotic Arm and Inverse Kinematics

The robotic arm mode simulates a 2 link planar arm with a simple gripper. The base of the arm is fixed near the bottom center of the screen. The user moves their index fingertip within a constrained workspace box, and this position is mapped to a target point for the end effector.

Inverse kinematics is solved using the geometry of a 2 link arm. The distance from the base to the target is computed and clamped to just inside the maximum reach, and the elbow angle is derived from the law of cosines. The shoulder angle is then computed using atan2. Once the joint angles are known, standard trigonometry is used to find the elbow and end effector coordinates.

The gripper is drawn as two short line segments extending from the end effector. When a pinch is detected, the segments move closer together and change color to indicate a closed gripper. When an open hand is detected, the segments spread apart again. This gives a clear visual link between the user's gestures and the simulated robot.

7. User Interface and Mode Design

The interface of HoloRig is styled to look like a holographic control console. A faint grid, dark overlay, glow circles, and a small HUD with FPS and labels are drawn on top of the camera feed. These elements do not affect the logic, but they make the demos easier to read and give a consistent visual identity.

The application moves through four main modes: an intro screen, a menu with three tiles, the 3D demo modes, and the robotic arm mode. A pinch anywhere on the intro screen advances to the menu. In the menu, each tile corresponds to a demo and can be selected by pinching on it. Inside each demo, a back button in the top left corner can be activated with a pinch gesture to return to the menu.

For clarity, a status line at the top left of the screen shows the current mode, and a legend line at the bottom summarises the active gestures, such as pinch to rotate and two hands to zoom. This helps new users understand what they can do without additional instructions.

8. Results and Observations

In testing, HoloRig ran smoothly on a standard laptop with an integrated webcam. The combination of cvzone, MediaPipe, NumPy, and OpenCV is efficient enough that the system maintains responsive frame rates while drawing all of the UI layers and computing 3D transformations.

Informal user trials suggested that the basic gestures were easy to understand. The 3D demos in particular are immediately engaging, because people can see a direct relationship between their hand movements and the hologram. The robotic arm mode also serves as a simple but clear demonstration of inverse kinematics.

Some practical observations:

- The grid background and dark overlay help the holographic lines stand out, even when the camera background is busy.
- Palm tilt based Z rotation feels more natural than relying only on drag based rotation.
- A small set of distance based gestures is sufficient for robust control in this context.
- Avoiding heavy visual effects such as particle trails helps preserve performance.

9. Code Overview and Explanation

This section highlights a few representative parts of the HoloRig code. The full implementation is contained in a single Python file, but these snippets capture the main structure and ideas.

```
# Main loop (simplified)
while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    frame = cv2.resize(frame, (win_w, win_h))

    hands, img = detector.findHands(frame, flipType=False)

    # extract key landmarks if at least one hand is present
    if hands:
        hand0 = hands[0]
        lm0 = hand0["lmList"]
        index_pt = lm0[8]
        thumb_pt = lm0[4]
        # ... compute pinch distance etc.

    # update FPS and time
    now = time.time()
    dt = now - prev_time
    prev_time = now

    # handle current mode
    if mode == "intro":
        img = draw_intro(img, t_since_mode)
    elif mode == "menu":
        img = draw_menu(img, t_since_mode)
    elif mode in ["3d_rings", "3d_core"]:
        # draw 3D object and handle gestures
        pass
    elif mode == "arm":
        # draw robotic arm and handle gestures
        pass

    cv2.imshow("HoloRig", img)
    if cv2.waitKey(1) & 0xFF == 27:
        break
```

This code sketch shows the structure of the main loop. Each frame, the program captures a frame from the camera, runs hand detection, updates timing variables, and then branches to the correct drawing and interaction logic based on the current mode. This keeps the overall flow easy to follow.

```
def project_points(points3d, rx, ry, rz, zoom_factor):
    cx, sx = math.cos(rx), math.sin(rx)
    cy, sy = math.cos(ry), math.sin(ry)
    cz, sz = math.cos(rz), math.sin(rz)

    Rx = np.array([[1, 0, 0],
                   [0, cx, -sx],
                   [0, sx, cx]], np.float32)
    Ry = np.array([[cy, 0, sy],
                   [0, 1, 0],
                   [-sy, 0, cy]], np.float32)
    Rz = np.array([[cz, -sz, 0],
```

```
                [sz,  cz, 0],
                [0,   0, 1]], np.float32)

    R = Rz @ Ry @ Rx
    rotated = (R @ points3d.T).T

    scale = 200 * zoom_factor
    rotated_scaled = rotated * scale

    cx_screen, cy_screen = win_w // 2, win_h // 2
    pts2d = []
    depths = []
    for x, y, z in rotated_scaled:
        sx2d = int(cx_screen + x)
        sy2d = int(cy_screen - y)
        pts2d.append((sx2d, sy2d))
        depths.append(z)
    return pts2d, depths
```

This helper function applies rotation matrices to all 3D points and then projects them into 2D pixel coordinates. It also returns a list of depth values, which is later used to adjust line brightness. Keeping this logic in one place makes it easy to reuse across both 3D demos.

```
def ik_2link(target_x, target_y):
    dx = target_x - base_x
    dy = target_y - base_y
    d = math.hypot(dx, dy)

    max_reach = L1 + L2 - 10
    d = min(d, max_reach)

    cos2 = (d*d - L1*L1 - L2*L2) / (2*L1*L2)
    cos2 = max(-1.0, min(1.0, cos2))
    angle2 = math.acos(cos2)

    k1 = L1 + L2 * math.cos(angle2)
    k2 = L2 * math.sin(angle2)
    angle1 = math.atan2(dy, dx) - math.atan2(k2, k1)

    ex = base_x + L1 * math.cos(angle1)
    ey = base_y + L1 * math.sin(angle1)
    end_x = ex + L2 * math.cos(angle1 + angle2)
    end_y = ey + L2 * math.sin(angle1 + angle2)

    return angle1, angle2, int(ex), int(ey), int(end_x), int(end_y)
```

This function implements the 2 link inverse kinematics. It first clamps the target distance to within reach, then uses the law of cosines to find the elbow angle, and finally computes the shoulder angle. The function returns both angles and the positions of the elbow and end effector, which are then used to draw the links and joints.

10. Applications and Future Work

Although HoloRig was originally built as a workshop demo for ORIGO 2025, the same techniques can be adapted to other domains such as AR and VR interaction prototypes, classroom visualisations of 3D math and robotics, and interactive exhibits at technical festivals. Future work may focus on adding more demos, refining the gesture set, and interfacing the software with a real robotic arm.

11. References

- OpenCV documentation: https://opencv.org/
- NumPy documentation: https://numpy.org/
- CVZone and HandTrackingModule: https://www.computervision.zone/
- MediaPipe Hands solution: Google AI documentation
- J. J. Craig, Introduction to Robotics: Mechanics and Control, Pearson.

Live Demo Video

Click the link below to watch the demo:

https://drive.google.com/file/d/1HM5tvKdJ276axRBrNijLc2xt1LMk5uRT/view?usp=sharing